ФГБОУ ВО «Пензенский государственный университет»
Политехнический институт
Факультет вычислительной техники
Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №4
по дисциплине «Логика и ОА в ИЗ»
на тему «Бинарное дерево поиска»

Выполнили: ст. гр. 25ВВВ2
Перетрухин О. С.
Павлов А. Д.
Проверили:
Юрова О. В.
Деев М. В.

Пенза 2026

Цель работы: изучить устройство бинарного дерева поиска; реализовать добавление, вывод и поиск элементов; исследовать обработку повторяющихся значений и оценить временную сложность поиска.

1. Построение дерева и поиск значения

Задание: реализовать бинарное дерево поиска, обеспечить ввод целых чисел, наглядный вывод структуры дерева и поиск заданного значения.

Каждый узел хранит целое число и два указателя на левое и правое поддеревья. Значения, меньшие текущего, помещаются слева, остальные — справа. Функция searchValue() рекурсивно выбирает одно поддерево, пока не найдёт искомое значение либо пустой указатель.

Листинг 1 — Программа построения дерева и поиска значения

```
#include <stdio.h>
#include <stdlib.h>
#include <windows.h>

#pragma execution_character_set("utf-8")

struct Node
{
    int data;
    Node* left;
    Node* right;
};

Node* createNode(int data)
{
    Node* node = (Node*)malloc(sizeof(Node));

    if (node == NULL)
    {
        printf("Ошибка выделения памяти.\n");
        exit(1);
    }

    node->data = data;
    node->left = NULL;
    node->right = NULL;
    return node;
}

Node* insertNode(Node* root, int data)
{
    if (root == NULL)
    {
        return createNode(data);
    }

    if (data < root->data)
    {
        root->left = insertNode(root->left, data);
    }
    else
    {
        root->right = insertNode(root->right, data);
    }

    return root;
}

void printTree(const Node* root, int level)
{
```

```
    if (root == NULL)
    {
        return;
    }

    printTree(root->right, level + 1);

    for (int i = 0; i < level; i++)
    {
        printf("    ");
    }

    printf("%d\n", root->data);
    printTree(root->left, level + 1);
}

const Node* searchValue(const Node* root, int value)
{
    if (root == NULL || root->data == value)
    {
        return root;
    }

    if (value < root->data)
    {
        return searchValue(root->left, value);
    }

    return searchValue(root->right, value);
}

void deleteTree(Node* root)
{
    if (root == NULL)
    {
        return;
    }

    deleteTree(root->left);
    deleteTree(root->right);
    free(root);
}

int main()
{
    SetConsoleCP(CP_UTF8);
    SetConsoleOutputCP(CP_UTF8);

    Node* root = NULL;
    int value;

    printf("Введите целые числа для построения дерева.\n");
    printf("Для завершения ввода введите -1.\n\n");

    while (true)
    {
        printf("Введите число: ");

        if (scanf_s("%d", &value) != 1)
        {
            printf("Ошибка ввода.\n");
            deleteTree(root);
            return 1;
        }

        if (value == -1)
        {
            break;
        }

        root = insertNode(root, value);
    }

    if (root == NULL)
    {
        printf("Дерево пусто.\n");
        return 0;
    }

    printf("\nПостроенное дерево:\n");
    printTree(root, 0);
```

```
    printf("\nВведите значение для поиска: ");

    if (scanf_s("%d", &value) != 1)
    {
        printf("Ошибка ввода.\n");
        deleteTree(root);
        return 1;
    }

    if (searchValue(root, value) != NULL)
    {
        printf("Значение %d найдено в дереве.\n", value);
    }
    else
    {
        printf("Значение %d не найдено в дереве.\n", value);
    }

    deleteTree(root);
    return 0;
}
```

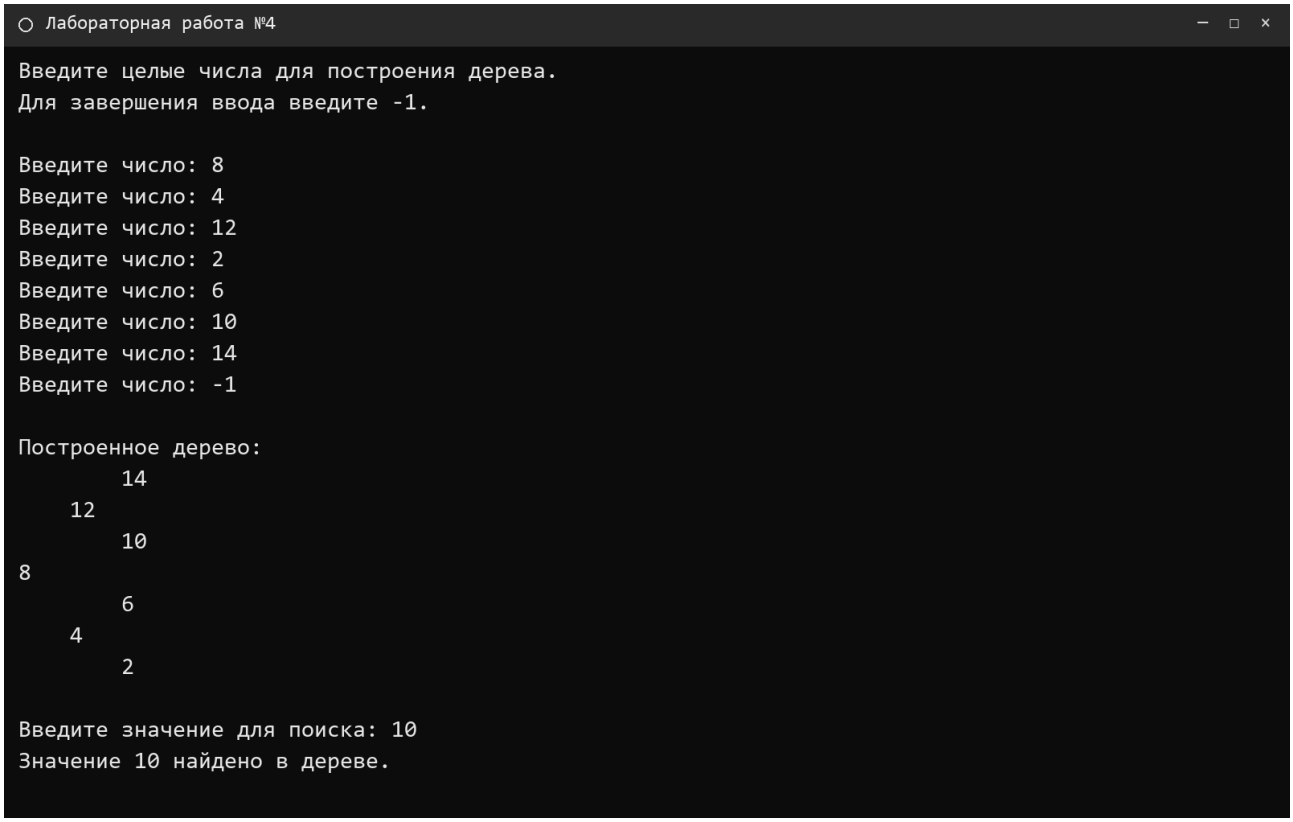


Рисунок 1 — Результат контрольного запуска задания 1

Из введённых значений построено бинарное дерево поиска. Значение 10 найдено в правом поддереве, что подтверждает правильность рекурсивного поиска.

2. Подсчёт повторяющихся значений

Задание: разрешить добавление одинаковых значений в бинарное дерево и определить количество вхождений заданного числа.

Повторяющиеся значения помещаются в правое поддерево. Функция countOccurrences() сравнивает искомое число со значением текущего узла. После совпадения счётчик увеличивается, а поиск продолжается справа, где могут находиться остальные равные элементы.

Листинг 2 — Программа подсчёта повторяющихся значений

```
#include <stdio.h>
#include <stdlib.h>
#include <windows.h>

#pragma execution_character_set("utf-8")

struct Node
{
    int data;
    Node* left;
    Node* right;
};

Node* createNode(int data)
{
    Node* node = (Node*)malloc(sizeof(Node));

    if (node == NULL)
    {
        printf("Ошибка выделения памяти.\n");
        exit(1);
    }

    node->data = data;
    node->left = NULL;
    node->right = NULL;
    return node;
}

Node* insertNode(Node* root, int data)
{
    if (root == NULL)
    {
        return createNode(data);
    }

    if (data < root->data)
    {
        root->left = insertNode(root->left, data);
    }
    else
    {
        root->right = insertNode(root->right, data);
    }

    return root;
}

void printTree(const Node* root, int level)
{
    if (root == NULL)
    {
        return;
    }

    printTree(root->right, level + 1);

    for (int i = 0; i < level; i++)
    {
        printf("    ");
```

```
    }

    printf("%d\n", root->data);
    printTree(root->left, level + 1);
}

int countOccurrences(const Node* root, int value)
{
    if (root == NULL)
    {
        return 0;
    }

    if (value < root->data)
    {
        return countOccurrences(root->left, value);
    }

    if (value > root->data)
    {
        return countOccurrences(root->right, value);
    }

    return 1 + countOccurrences(root->right, value);
}

void deleteTree(Node* root)
{
    if (root == NULL)
    {
        return;
    }

    deleteTree(root->left);
    deleteTree(root->right);
    free(root);
}

int main()
{
    SetConsoleCP(CP_UTF8);
    SetConsoleOutputCP(CP_UTF8);

    Node* root = NULL;
    int value;

    printf("Введите целые числа для построения дерева.\n");
    printf("Повторяющиеся значения разрешены.\n");
    printf("Для завершения ввода введите -1.\n\n");

    while (true)
    {
        printf("Введите число: ");

        if (scanf_s("%d", &value) != 1)
        {
            printf("Ошибка ввода.\n");
            deleteTree(root);
            return 1;
        }

        if (value == -1)
        {
            break;
        }

        root = insertNode(root, value);
    }

    if (root == NULL)
    {
        printf("Дерево пусто.\n");
        return 0;
    }

    printf("\nПостроенное дерево:\n");
    printTree(root, 0);

    printf("\nВведите значение для подсчёта: ");

    if (scanf_s("%d", &value) != 1)
    {
```

```
        printf("Ошибка ввода.\n");
        deleteTree(root);
        return 1;
    }

    const int count = countOccurrences(root, value);
    printf("Число вхождений значения %d: %d\n", value, count);

    deleteTree(root);
    return 0;
}
```

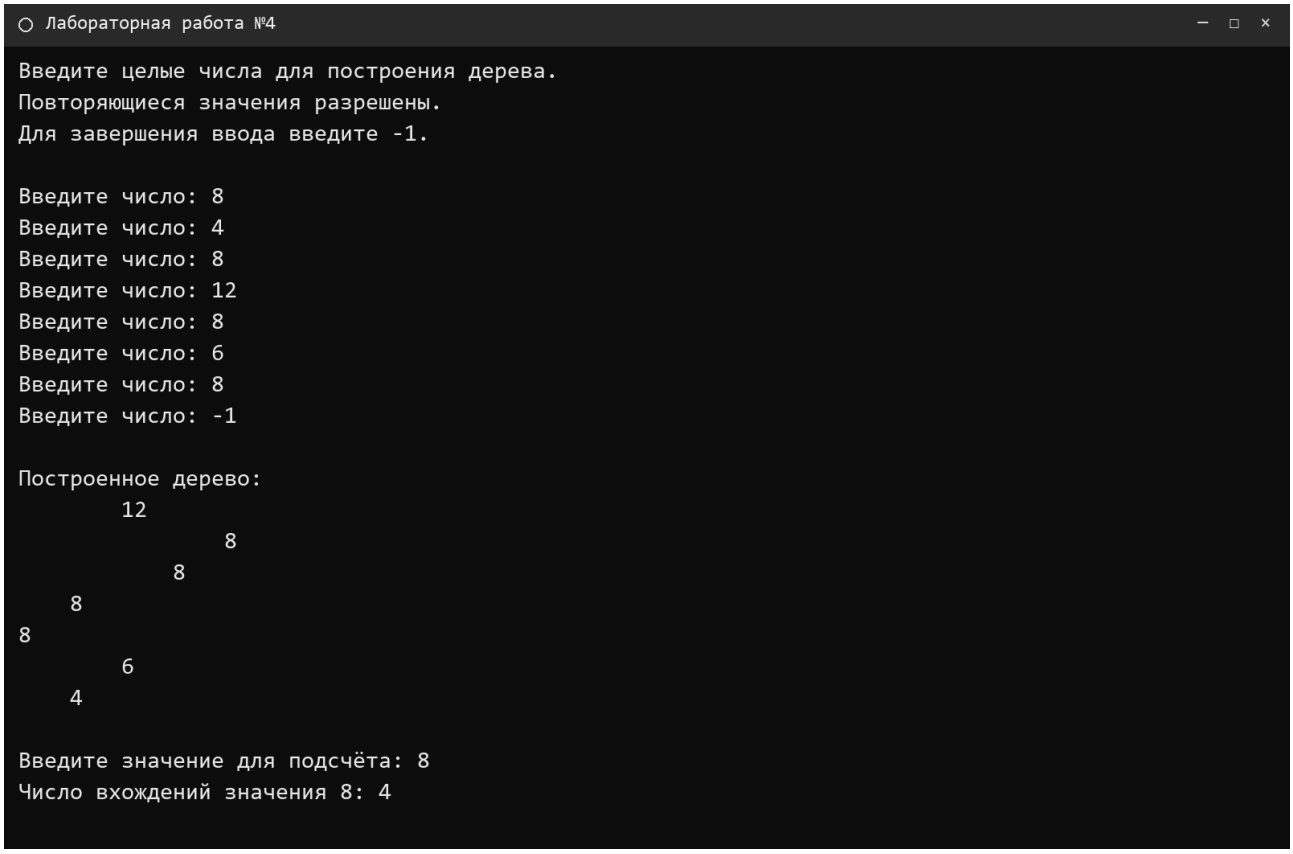


Рисунок 2 — Результат контрольного запуска задания 2

Число 8 введено четыре раза. Программа сохранила все повторения в дереве и вернула количество вхождений, равное четырём.

3. Запрет добавления повторяющихся значений

Задание: изменить добавление элементов так, чтобы одинаковые значения не помещались в дерево, а пользователю выводилось соответствующее сообщение.

В функции insertNode() отдельно обрабатываются три результата сравнения. Меньшее значение направляется в левое поддерево, большее — в правое. При равенстве новый узел не создаётся, структура дерева не изменяется и выводится сообщение «Такое значение уже существует!».

Листинг 3 — Программа без добавления повторяющихся значений

```
#include <stdio.h>
#include <stdlib.h>
#include <windows.h>

#pragma execution_character_set("utf-8")

struct Node
{
    int data;
    Node* left;
    Node* right;
};

Node* createNode(int data)
{
    Node* node = (Node*)malloc(sizeof(Node));

    if (node == NULL)
    {
        printf("Ошибка выделения памяти.\n");
        exit(1);
    }

    node->data = data;
    node->left = NULL;
    node->right = NULL;
    return node;
}

Node* insertNode(Node* root, int data)
{
    if (root == NULL)
    {
        return createNode(data);
    }

    if (data < root->data)
    {
        root->left = insertNode(root->left, data);
    }
    else if (data > root->data)
    {
        root->right = insertNode(root->right, data);
    }
    else
    {
        printf("Такое значение уже существует!\n");
    }

    return root;
}

void printTree(const Node* root, int level)
{
    if (root == NULL)
    {
        return;
```

```
    }

    printTree(root->right, level + 1);

    for (int i = 0; i < level; i++)
    {
        printf("    ");
    }

    printf("%d\n", root->data);
    printTree(root->left, level + 1);
}

int countOccurrences(const Node* root, int value)
{
    if (root == NULL)
    {
        return 0;
    }

    if (value < root->data)
    {
        return countOccurrences(root->left, value);
    }

    if (value > root->data)
    {
        return countOccurrences(root->right, value);
    }

    return 1 + countOccurrences(root->right, value);
}

void deleteTree(Node* root)
{
    if (root == NULL)
    {
        return;
    }

    deleteTree(root->left);
    deleteTree(root->right);
    free(root);
}

int main()
{
    SetConsoleCP(CP_UTF8);
    SetConsoleOutputCP(CP_UTF8);

    Node* root = NULL;
    int value;

    printf("Введите целые числа для построения дерева.\n");
    printf("Повторяющиеся значения не добавляются.\n");
    printf("Для завершения ввода введите -1.\n\n");

    while (true)
    {
        printf("Введите число: ");

        if (scanf_s("%d", &value) != 1)
        {
            printf("Ошибка ввода.\n");
            deleteTree(root);
            return 1;
        }

        if (value == -1)
        {
            break;
        }

        root = insertNode(root, value);
    }

    if (root == NULL)
    {
        printf("Дерево пусто.\n");
        return 0;
    }
```

```
    printf("\nПостроенное дерево:\n");
    printTree(root, 0);

    printf("\nВведите значение для подсчёта: ");

    if (scanf_s("%d", &value) != 1)
    {
        printf("Ошибка ввода.\n");
        deleteTree(root);
        return 1;
    }

    const int count = countOccurrences(root, value);
    printf("Число вхождений значения %d: %d\n", value, count);

    deleteTree(root);
    return 0;
}
```

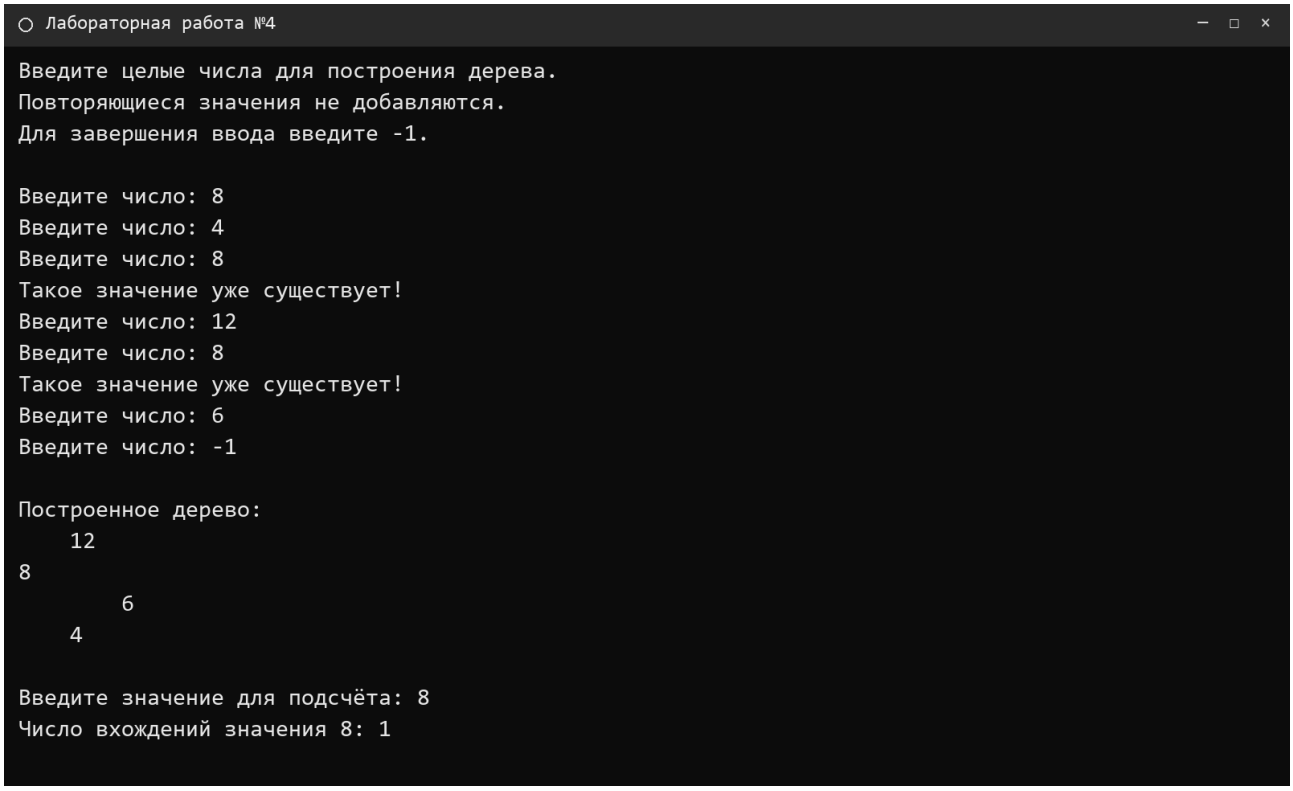


Рисунок 3 — Результат контрольного запуска задания 3

При повторном вводе числа 8 программа дважды вывела предупреждение. В итоговом дереве сохранился только один узел со значением 8, поэтому подсчёт вернул единицу.

4. Оценка временной сложности поиска

Задание: определить временную сложность функции countOccurrences() для сбалансированного и вырожденного бинарного дерева поиска.

В функции countOccurrences() в начале проверяется, является ли текущий узел пустым (if (root == NULL)). Затем искомое значение сравнивается со значением текущего узла. В зависимости от результата функция переходит в левое или правое поддерево. На каждом шаге выполняется постоянное количество операций — O(1).

Общая сложность зависит от высоты дерева h и составляет O(h). Для сбалансированного дерева сложность равна O(log n), а для вырожденного в цепочку — O(n).

Вывод по заданию: в данной программе сложность поиска составляет O(log n) для сбалансированного дерева и O(n) в худшем случае, поскольку балансировка не предусмотрена.

Вывод: в ходе лабораторной работы реализовано бинарное дерево поиска с рекурсивными операциями добавления, вывода, поиска и подсчёта вхождений. Проверены два способа обработки повторов: сохранение одинаковых значений и запрет их добавления. Контрольные запуски подтвердили правильность программ. Время поиска определяется высотой дерева и изменяется от O(log n) для сбалансированной структуры до O(n) в худшем случае.